

Capture the Flag 5: Make the System Interactive

Goal: Transform your Student Directory into an interactive application where administrators can favorite students, edit information, and remove records using Flutter interactions and state management.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

Flag 0 — Create Your Feature Branch

Objective: Create a new feature branch before adding interactions.

Create:

`feature/student-interactions`

Screenshot:

- Terminal showing your current branch

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\kevin\IT314\IT314AB_Villacorte> git branch
* feature/student-list
  main
PS C:\Users\kevin\IT314\IT314AB_Villacorte> git status
On branch feature/student-list
Your branch is up to date with 'origin/feature/student-list'.

nothing to commit, working tree clean
PS C:\Users\kevin\IT314\IT314AB_Villacorte> git switch -c feature/student-interactions
Switched to a new branch 'feature/student-interactions'
PS C:\Users\kevin\IT314\IT314AB_Villacorte> git branch
* feature/student-interactions
  feature/student-list
  main
PS C:\Users\kevin\IT314\IT314AB_Villacorte> |
```

🚩 Flag 1 — Wake Up the Buttons

Add at least two buttons inside each Student Card.

Example actions:

- Favorite
- Edit

Objective: Make your Student Card respond to button presses using `onPressed`.

Right now your buttons are only decorations.

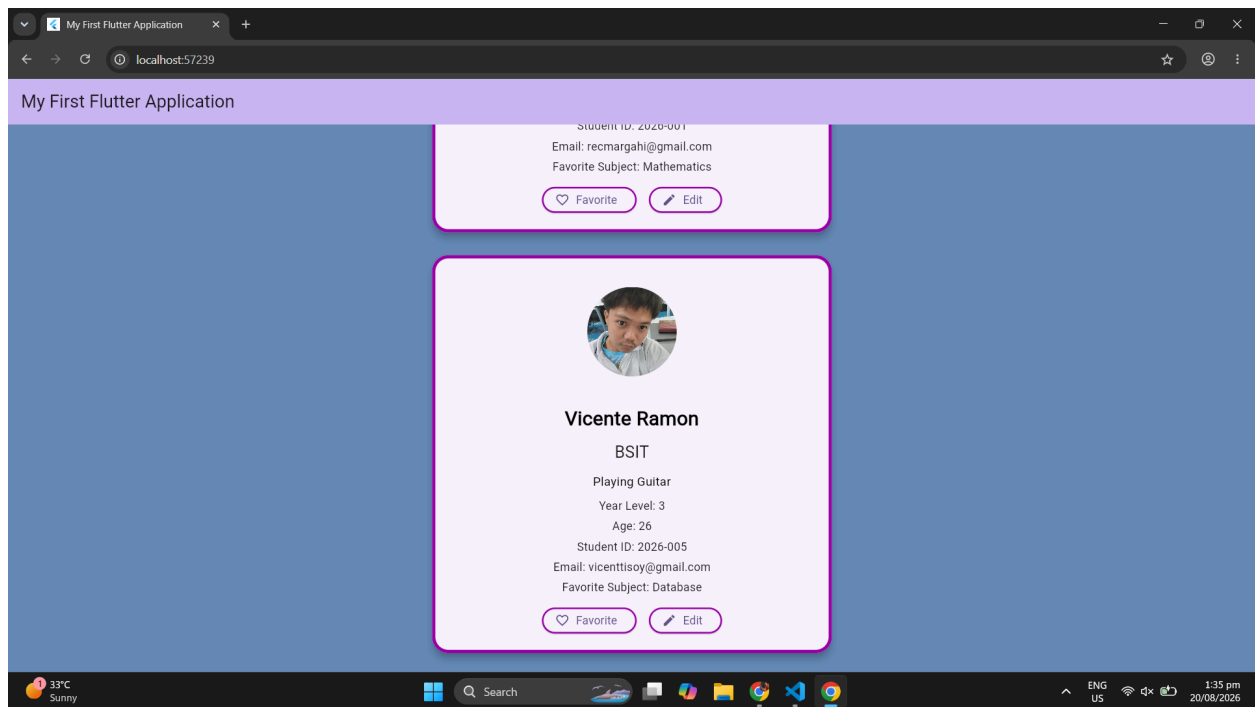
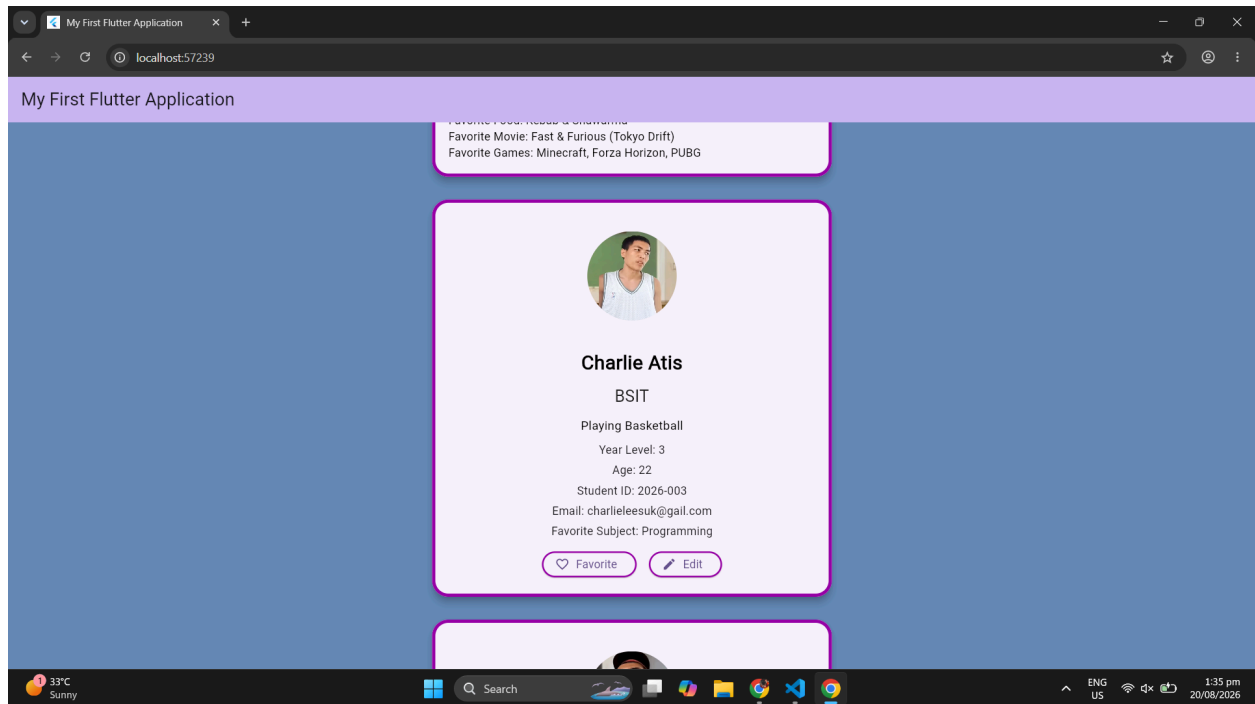
Research how Flutter's `onPressed` works.

The buttons do not need to perform their final behavior yet. They simply need to respond when pressed.

Test

When a button is pressed, your app should visibly react in some way (such as printing a message or showing a temporary notification).

Screenshot:



- Student Card with buttons

```

const SizedBox(height: 15),

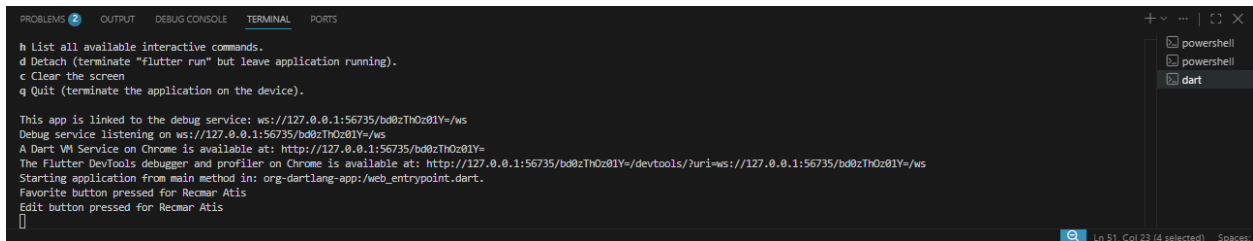
Row(
  mainAxisAlignment: MainAxisAlignment.center,
  children: [
    ElevatedButton.icon(
      onPressed: () {
        print('Favorite button pressed for ${students[index]['name']}');
      },
      icon: const Icon(Icons.favorite_border),
      label: const Text('Favorite'),
      style: ElevatedButton.styleFrom(
        side: const BorderSide(
          color: Color.fromARGB(255, 152, 0, 169),
          width: 2,
        ) // BorderSide
      )
    ), // ElevatedButton.icon

    const SizedBox(width: 15),

    ElevatedButton.icon(
      onPressed: () {
        print('Edit button pressed for ${students[index]['name']}');
      },
      icon: const Icon(Icons.edit),
      label: const Text('Edit'),
      style: ElevatedButton.styleFrom(
        side: const BorderSide(
          color: Color.fromARGB(255, 152, 0, 169),
          width: 2,
        ) // BorderSide
      )
    ), // ElevatedButton.icon
  ],
), // Row

```

- **onPressed** inside your code



The screenshot shows the Flutter IDE interface with the terminal open. The terminal displays the following output:

```

h List all available interactive commands.
d Detach (terminate "Flutter run" but leave application running).
c Clear the screen
q Quit (terminate the application on the device).

This app is linked to the debug service: ws://127.0.0.1:56735/bd0zTh0z01Y=/ws
Debug service listening on ws://127.0.0.1:56735/bd0zTh0z01Y=/ws
A Dart VM Service on Chrome is available at: http://127.0.0.1:56735/bd0zTh0z01Y=/ws
The Flutter DevTools debugger and profiler on Chrome is available at: http://127.0.0.1:56735/bd0zTh0z01Y=/devtools/?uri=ws://127.0.0.1:56735/bd0zTh0z01Y=/ws
Starting application from main method in: org-dartlang-app/web_entrypoint.dart.
Favorite button pressed for Recmar Atis
Edit button pressed for Recmar Atis

```

- Button reacting or terminal message

🚩 Flag 2 — Touch Anywhere

Objective: Make the entire Student Card tappable using **onTap**.

Buttons aren't the only interactive elements.

Research Flutter's **onTap**.

Wrap your Student Card so tapping anywhere on the card triggers an interaction.

Test

The user should be able to:

- Tap the Favorite button
- Tap the Edit button
- Tap anywhere else on the Student Card

Each interaction should be recognized separately. (One function for the 2 buttons, another for the Student Card)

Screenshot:

```
child: GestureDetector(
  onTap: () {
    print(
      'Student Card tapped: ${students[index]['name']}',
    );
  },
),
```

- `onTap` in your code

```
Student Card tapped: Vicente Ramon
Student Card tapped: Recmar Atis
Student Card tapped: Larfie Atis
Student Card tapped: Islam Makhachev
Student Card tapped: Clifford Atis
Student Card tapped: Charlie Atis
[]
```

- Student Card being tapped

🚩 Flag 3 — Discover `setState`

Objective: Update the screen immediately using `setState`.

So far your app reacts. But does the UI actually change?

Research Flutter's `setState`.

Use it so pressing a button immediately updates something visible on the screen.

Examples include:

- Changing text
- Changing an icon
- Updating a counter

The important part is understanding that `setState` tells Flutter to rebuild the UI.

Test

The screen should visibly change immediately after pressing the button.

Screenshot:

A screenshot of a code editor with a dark background and light-colored text. The code is written in a programming language that uses `ElevatedButton` and `setState`, likely Flutter/Dart. The code defines a `Favorite button` widget. It uses `ElevatedButton.icon` with an `onPressed` callback that calls `setState` to increment `buttonPressed`.

```
// Favorite button
ElevatedButton.icon(
  onPressed: () {
    setState(() {
      buttonPressed++;
    });
  },
```

- `setState` in your code



Larfie Atis

BSMARE

Playing Basketball

Year Level: 3

Age: 21

Student ID: 2026-002

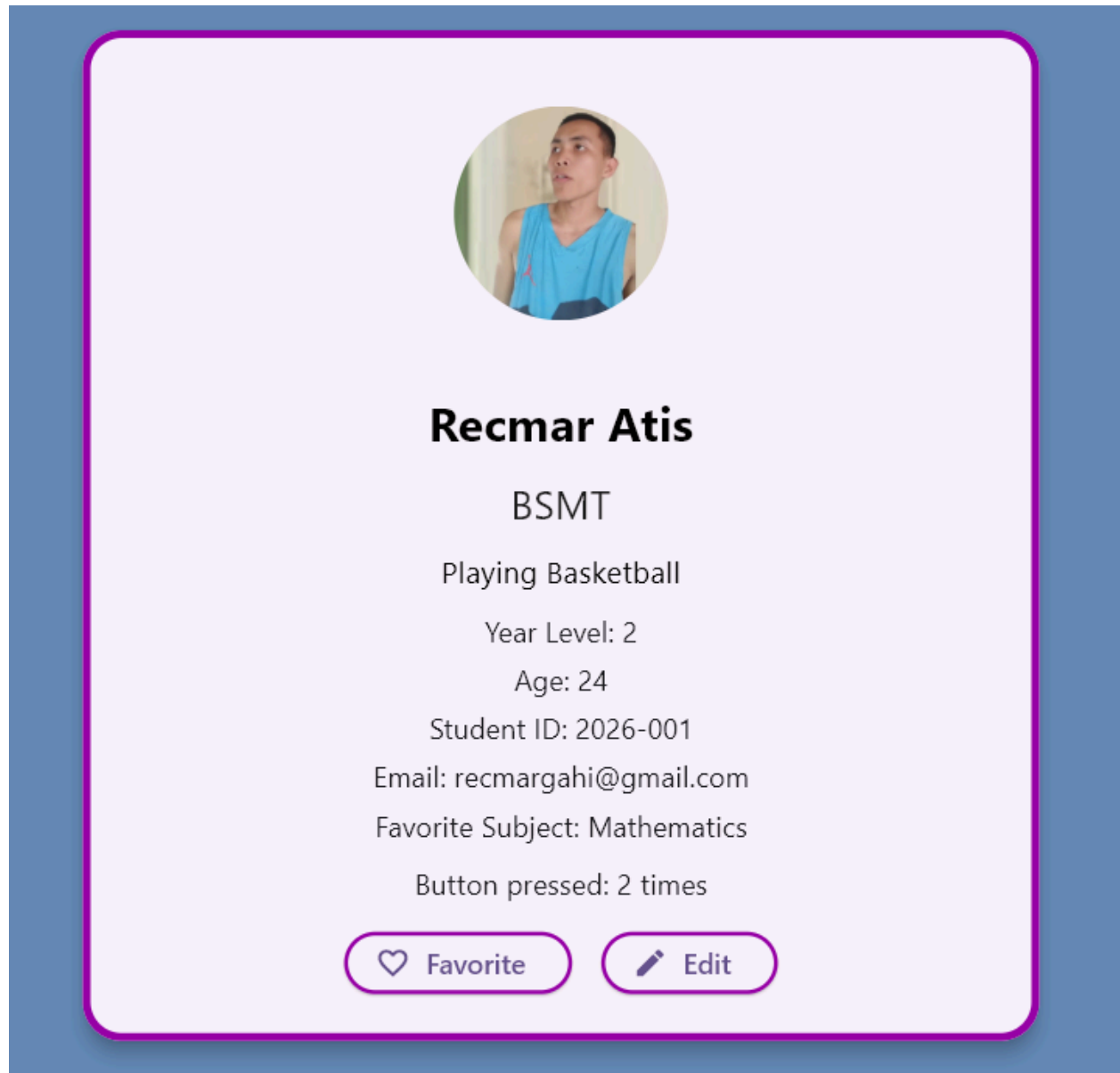
Email: larfiethefluff@gmail.com

Favorite Subject: Science

Button pressed: 0 times

♡ Favorite

✎ Edit



- Before and after interaction

🚩 Flag 4 — Favorite Students

Objective: Let administrators mark students as favorites.

Schools often need to highlight important records.

Add a favorite feature that changes a student's appearance when marked.

Possible visual changes include:

- Filled heart/star
- Different card color
- Favorite label

Each student's favorite status should work independently.

Test

- Favorite one student.
- Other students should remain unchanged.
- Favorite another student.

Both should keep their own state.

Screenshot:



Islam Makhachev

BSIT

Playing Games

Year Level: 3

Age: 20

Student ID: 2026-006

Email: makhachev@gmail.com

Favorite Subject: Web Development

♥ Favorited

✎ Edit

- Favorited student



Clifford Atis

BSMT

Watching SpiderMan

Year Level: 3

Age: 21

Student ID: 2026-004

Email: cliffordgod@gmail.com

Favorite Subject: English

♡ Favorite

✎ Edit

- Non-favorited student

♡ Favorite

♥ Favorited

- Favorite icon changing

🚩 Flag 5 — Remove a Student

Objective: Delete a student from the directory.

Research how to remove an item from a Dart `List`.

Add a Delete action that removes a student from your list.

The UI should immediately update after deletion.

Test

Before:

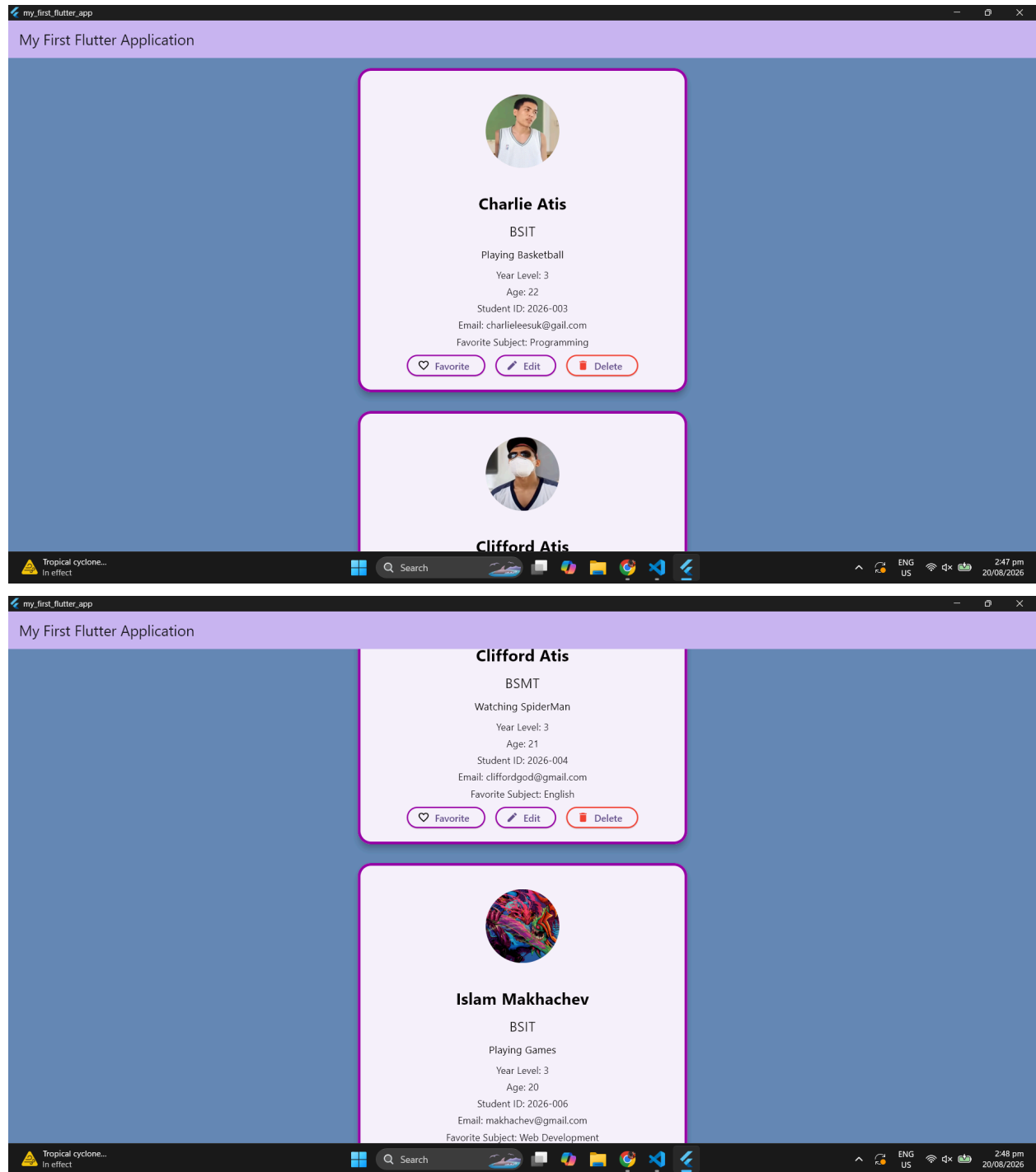
- Student A
- Student B
- Student C

After deleting Student B:

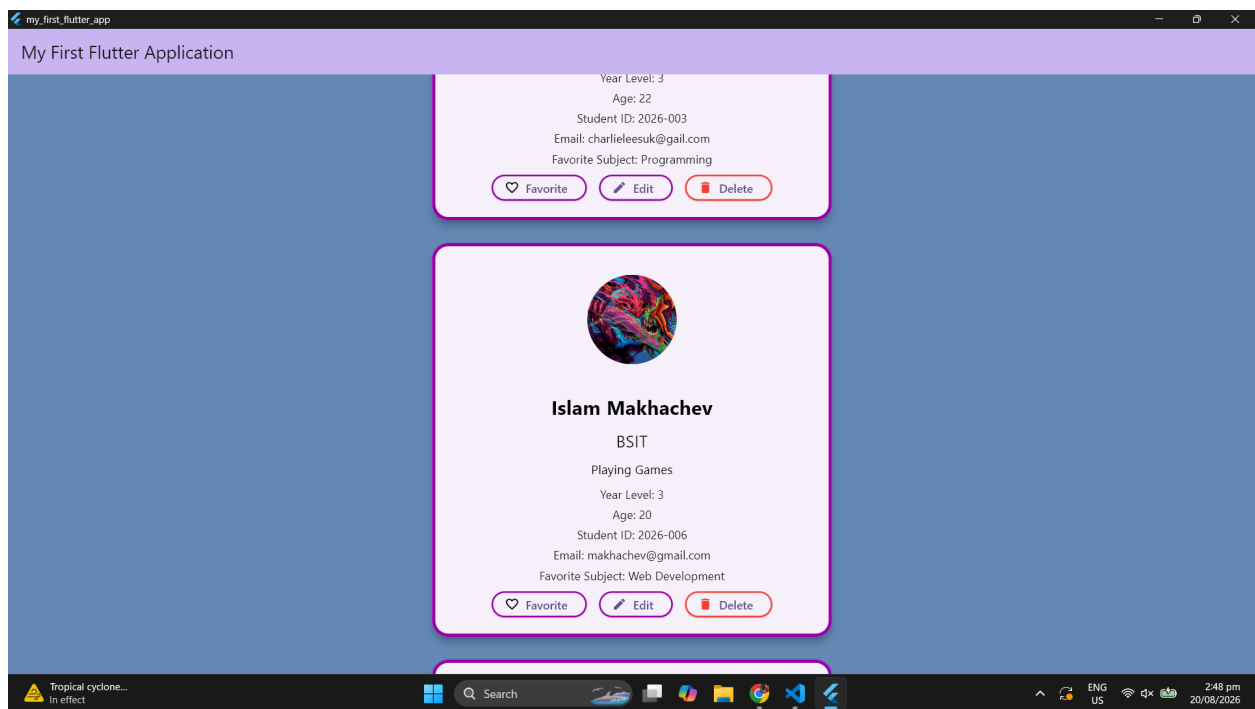
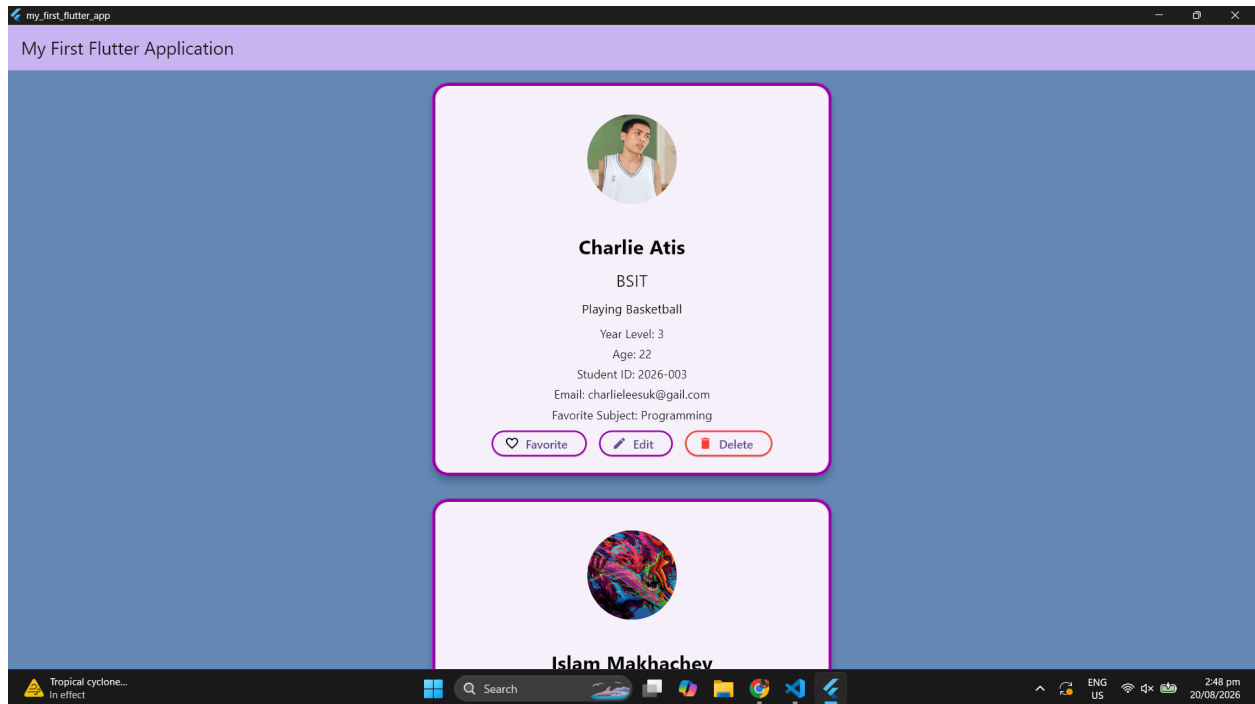
- Student A
- Student C

No empty spaces should remain.

Screenshot:



- Student before deletion



- Student after deletion

🚩 Flag 6 — Edit Trigger

Objective: Prepare your app for editing student information.

In the next CTF you'll build a full Edit Screen.

For now, create an Edit trigger.

Research simple ways Flutter can display temporary editing interfaces.

Examples include:

- `AlertDialog`
- `SnackBar`
- Bottom Sheet

The goal is simply to launch an edit action, not build the complete editor yet.

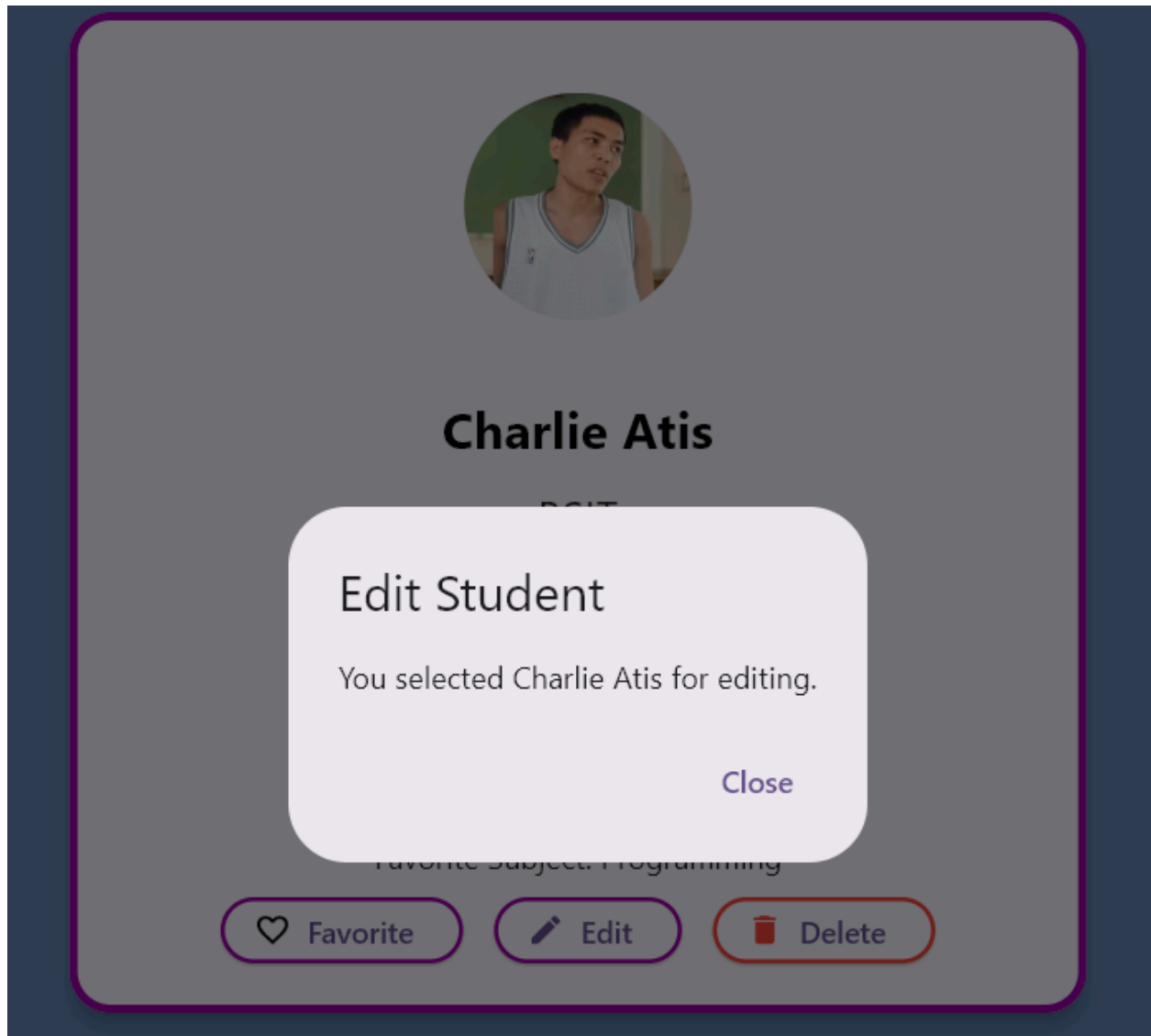
Test

Pressing Edit should open your chosen interface.

Screenshot:



- Edit button



- Opened dialog, snackbar, or bottom sheet

🚩 Flag 7 — Multiple Actions, Independent State

Objective: Make every Student Card manage interactions correctly.

This is your final interaction challenge.

Verify that every student behaves independently.

Example scenario:

- Favorite Student A.

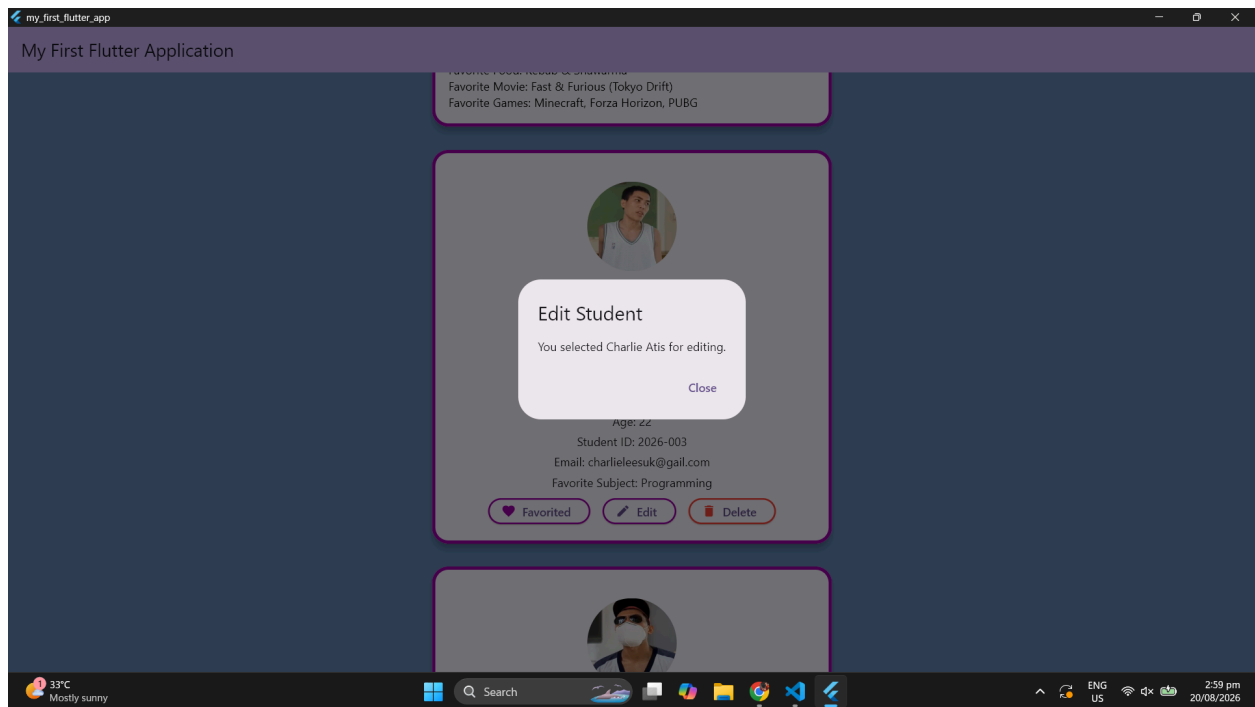
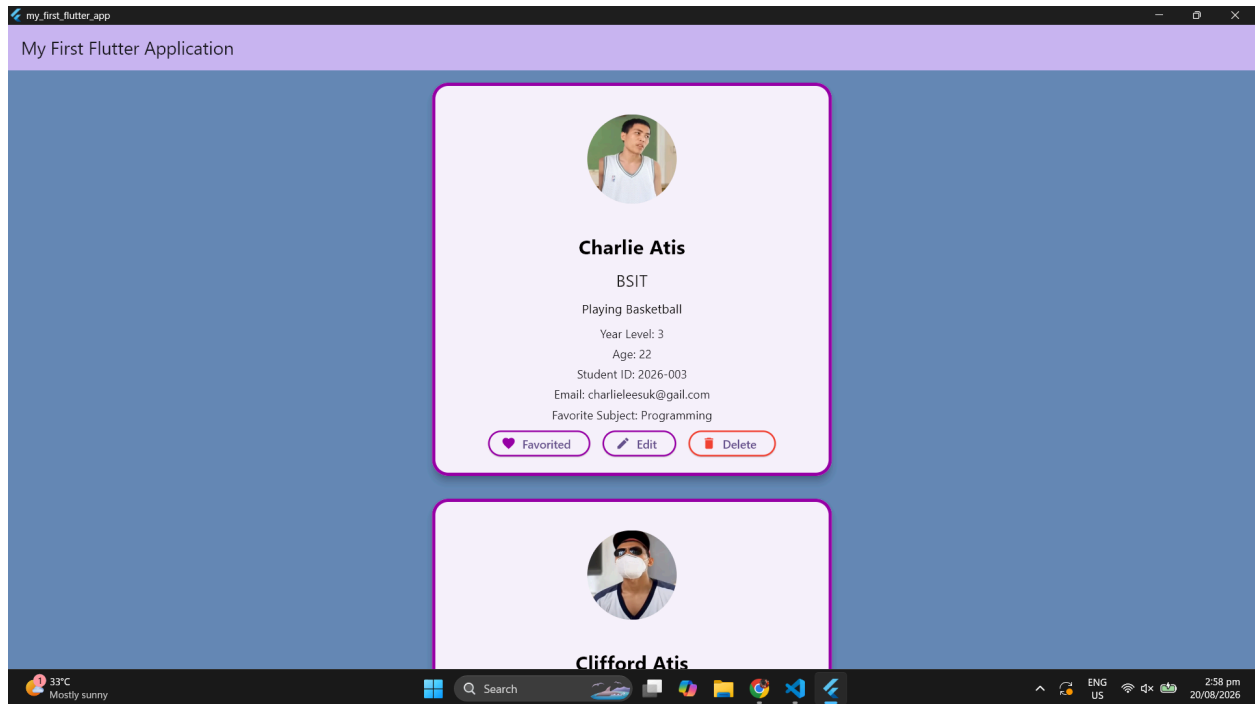
- Delete Student C.
- Open Edit for Student B.

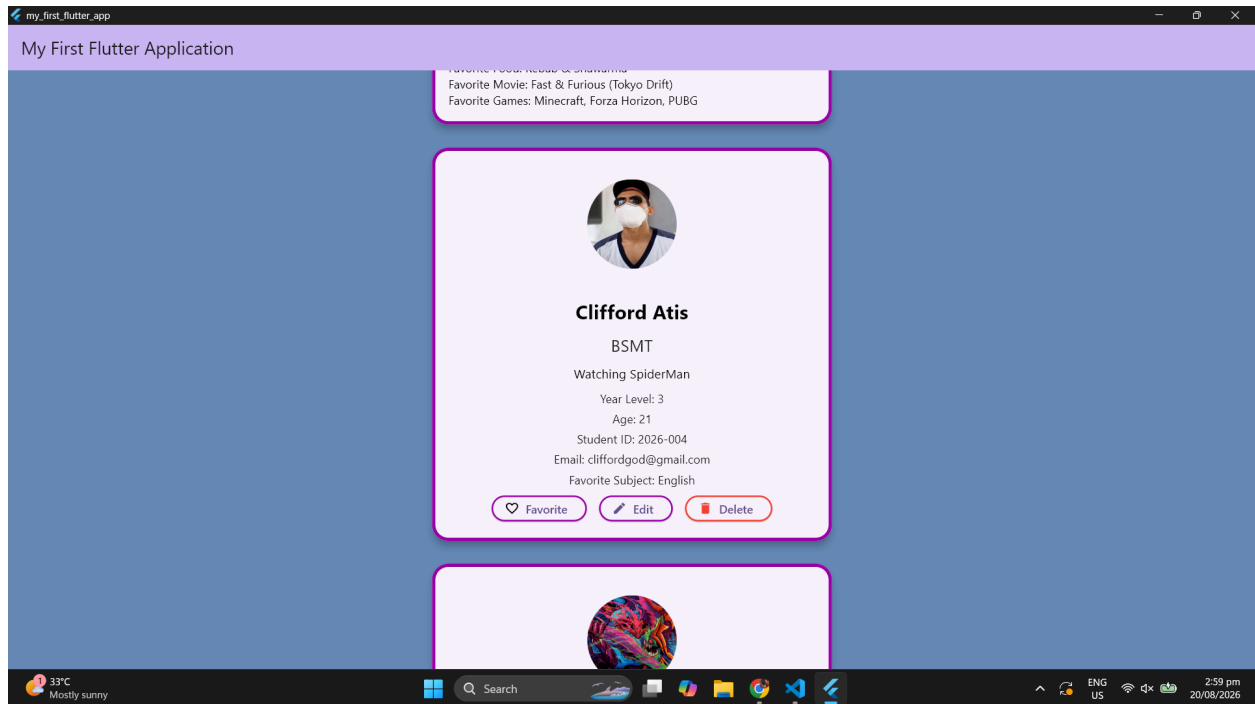
Your application should continue functioning correctly without affecting unrelated students.

Test Checklist

- Favorite works independently.
- Delete updates the list immediately.
- Edit still opens correctly.
- Remaining students keep their own states.

Screenshot:





- Multiple interactions working together
- Application still functioning normally

🚩 Flag 8 — Save Your Progress

Objective: Commit and push your completed Interactive Student Directory.

Use a proper Git commit message.

Branch:

`feature/student-interactions`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit